

Enoncés TDs et TP / Angular v ≥ 17 , partie 4

1. TP approfondissement du "routing angular"

1.1. Sous niveau de routes (en mode "children")

Le composant intermédiaire "basic" (comportant les sous composants "tva" et "calculatrice") a déjà été codé en 3 variantes :

1. avec `<app-calculatrice>` et `<app-tva>` toujours affichés
2. avec intégration dans le composant réutilisable `<app-toggle-panel>`
3. avec basculement via onglets dynamiques de l'extension material

On va maintenant mettre en œuvre une quatrième variante utilisant un sous-niveau "children : []" du routing angular et une nouvelle utilisation de `<router-outlet>`.

- ajouter `<router-outlet></router-outlet>` dans **basic.component.html**
et commenter éventuellement les anciens blocs `<app-toggle-panel>` ou `<mat-tab-group><mat-tab>...</>`
ou `<app-tva>` et `<app-calculatrice>`

- ajouter dans **basic.component.html**

```
<a routerLink="/ngr-basic/calculatrice" >calculatrice</a><br/>
<a routerLink="/ngr-basic/tva" >tva</a><br/>
```

NB : ajouter si besoin `RouterLink` , `RouterOutlet` dans la partie `imports:[]` de **BasicComponent** en version standalone

- ajuster un bloc de configuration de ce type dans **app-routes.ts** ou bien **app-routing.module.ts**:

```
{ path: 'ngr-basic', component: BasicComponent ,
  children: [
    { path: 'tva', component: TvaComponent },
    { path: 'calculatrice', component: ...Component },
    { path: '', redirectTo: 'tva', pathMatch: 'prefix' }
  ]
}
```

- tester le nouveau comportement de l'application

1.2. Avec paramètres en fin de routes

Seconde version (avec 2 modes sur calculatrice : "simple" et "sophistiquée"):

- ajouter (dans `calculatrice.component.ts`):
modeChoisi = "simple"; //ou "sophistiquée"

- ajouter (dans `calculatrice.component.html`) un paramétrage pour faire en sorte que le bouton poussoir **"multiplication"** ne soit affiché que si **modeChoisi=='sophistiquée'**

- ajuster les liens hypertextes suivants (`basic.component.html`):

```
<a routerLink="/ngr-basic/calculatrice/simple" >calculatrice simple</a> &nbsp; ;  
<a routerLink="/ngr-basic/calculatrice/sophistiquee" >calculatrice sophistiquee</a><br/>
```

- ajuster la route imbriquée au sein app-routing.module.ts:

```
children: [ ...,  
  { path: 'calculatrice/:mode', component: CalculatriceComponent }, ...  
]
```

- ajouter et ajuster (dans calculatrice.component.ts):

```
constructor(route : ActivatedRoute) {  
  //NB: { path: 'calculatrice/:mode', ... },  
  route.params.subscribe(  
    (params:Params)=> { this.modeChoisi = params['mode'];}  
  );  
}
```

- tester le nouveau comportement de l'application

1.3. Mettre en œuvre un gardien (Tp facultatif)

- Ce gardien devra bloquer une route (ex : devise) tant que l'utilisateur n'est pas connecté
- On effectuera si besoin une redirection vers une page de login ou bien une page d'erreur expliquant le problème

Indications :

ng g component not-authorized *--type component*

répertoire **src/app/common/guard**

ng g guard auth

choisir "canActivate"

2. TD début de déploiement d'une appli angular

2.1. Petit test en mode "code angular sur serveur http"

npm install -g http-server

ng build

ou bien

ng build --ssr=false --prerender=false (si extension @angular/ssr déjà activée)

http-server --proxy https://www.d-defrance.fr -c-1 dist/my-app/browser

ou bien

http-server --proxy https://www.d-defrance.fr -c-1 dist/my-app (selon version d'angular)

URL de test : <http://localhost:8080>

NB : Avec du temps et des moyens suffisants , on faire mieux avec **nginx + docker** .

2.2. Petit test (long et facultatif) en mode SSR/nodeJs

NB : ce Tp n'est généralement effectué qu'au sein d'une formation "angular avancé"

ng add @angular/ssr (si pas déjà fait)

Prise en compte du contexte via éventuels ajouts de *if (isPlatformBrowser(this.platform)) { ...}* si nécessaire

npm run build

npm run serve:ssr:my-app

NB : Avec du temps et des moyens suffisants , on peut intégrer cela dans un contexte *nginx* + *docker* .

3. TP et TD sur tests au sein d'angular

3.1. TD facultatif : test "end-to-end" avec cypress et angular

Ajustement du code "angular" pour faciliter l'écriture des tests :

- Au sein de `basic.component.html` , faire en sorte qu'il n'y ait qu'une seule version du composant calculatrice (par exemple idéalement via `<router-outlet>` et routing avec url en `/ngr-basic/calculatrice/...` ou bien `<app-calculatrice>` en direct mais pas les deux)
- Au sein de `calculatrice.component.html` ajouter si besoin :
 - `name="a"` sur la balise `<input [(ngModel)]="a" />`
 - `name="b"` sur la balise `<input [(ngModel)]="b" />`
 - `id="spanRes"` sur la balise `<span>` englobant `{{res}}` **sans espace**

Installation de la technologie cypress dans le projet angular :

```
npm install --save-dev cypress
```

ou bien tout simplement `npm install` si est cypress déjà présent dans le fichier `package.json`

Attention : Prévoir de longues minutes de téléchargement ...

Réglages pour éviter un conflit avec angular :

Au sein d'un projet angular , il faut ajouter ceci à la fin du fichier `tsconfig.json` de manière à ce que l'installation de cypress ne perturbe pas les tests unitaires d'angular :

```
{
  ...
  "compilerOptions": { ....
  },
  "angularCompilerOptions": { ...
  },
  "exclude": [
    "cypress.config.ts",
    "cypress/**/*.ts"
  ]
}
```

Premier lancement de cypress :

```
npx cypress open
```

Remarque : Le **premier lancement** (avec choix **E2E testing** et **validation création config**) va permettre la création de `cypress.json` , du sous-répertoire `cypress` et de certains sous-répertoires avec quelques exemples

Après un lancement de `cypress open` , il faut en théorie choisir un test à lancer au sein de la fenêtre de sélection. Nous devons cependant écrire préalablement notre propre test. À arrêter (en fermant cette fenêtre et/ou "Ctrl-C") et à relancer ultérieurement.

Au sein du répertoire cypress/e2e , ajouter ce fichier :

myTest.spec.cy.js

```
//NB: il faut préalablement lancer ng-serve ou autre
describe('My Angular Tests', () => {
  it('good conversion', () => {

    //partir de index.html
    cy.visit("http://localhost:4200/index.html")

    //cliquer sur le lien comportant 'basic'
    cy.contains('basic').click()
    cy.wait(50)
    // Should be on a new URL which includes 'basic'
    cy.url().should('include', 'basic')

    cy.get('a[href="/ngr-basic/calculatrice/simple"]').click()

    // Get an input, type data into it
    //and verify that the value has been updated
    cy.get('input[name="a"]')
      .clear()
      .type('9')
      .should('have.value', '9')

    cy.get('input[name="b"]')
      .clear()
      .type('6')
      .should('have.value', '6')

    //declencher click sur bouton soustraction
    cy.get('input[type="button"][value="-"]')
      .click()

    //vérifier que la zone d'id spanRes comporte le texte '3'
    cy.get('#spanRes')
      .should('have.text', '3')

  })
})
```

Ajuster ci besoin le code de ce fichier pour garantir une bonne cohérence avec le code à tester.

* Vérifier que l'application angular est bien démarrée (ex : ng serve préalablement lancé).

* Relancer (si besoin) , **npx cypress open**

puis sélectionner le test à lancer (E2E testing , choix navigateur, choix du test)

3.2. Ajustement du code de TvaComponent pour préparer des tests unitaires cohérents

Au sein de `src/app/common/service`
générer un nouveau service de calcul de tva via
ng g service tva --type service

Ajouter dans **TvaService** les méthodes **tva** et **ttc** suivantes :

```
tva(ht:number ,taux_tva_pct:number){
  return ht * taux_tva_pct/100.0
}

ttc(ht:number ,taux_tva_pct:number){
  return ht * (1 + taux_tva_pct/100.0)
}
```

Injecter ensuite **ServiceTva** au sein de **TvaComponent** (via constructor ou `inject()`)
Puis réajuster les calculs de tva au sein de **TvaComponent** en s'appuyant sur le service injecté.
Par exemple :

```
tvaService = inject(TvaService)

onCalculerTvaTtc(){
  this.tva = this.tvaService.tva(this.ht, this.tauxTvaPct);
  this.ttc = this.tvaService.ttc(this.ht, this.tauxTvaPct);
}
ou bien
tva = computed(()=>this.tvaService.tva(this.ht(), this.tauxTvaPct()))
ttc =computed(()=> this.tvaService.ttc(this.ht(), this.tauxTvaPct()));
```

Du côté **tva.component.html** :

- ajouter si besoin **name="ht"** sur `<input .../>` à coté de `[(ngModel)]="ht"`
- ajouter **name="tauxTvaPct"** ou **name="taux"** sur `<select .../>` ou `<input .../>` à coté de `[(ngModel)]="tauxTvaPct"`

3.3. TP : test d'un service simple (TvaService)

Au sein du fichier **tva.service.spec.ts** :

- ajouter les méthodes de tests suivantes :

```
it('tva(200,20)==40', () => {
  expect(.....).toBe(40);
});

it('ttc(200,20)==240', () => {
  expect(.....).toBe(240);
});
```

Compléter le code et lancer le test unitaire en lançant la commande suivante :

```
ng test --include=**/tva.service.spec.ts
```

Corriger si besoin certaines erreurs de manière à obtenir un résultat de ce genre :

Karma v 6.4.4 - connected; test: complete;

Chrome 137.0.0.0 (Windows 10) is idle

Jasmine 4.6.1

3 specs, 0 failures, randomized with seed 11818

```
TvaService
  • tva(200,20)==40
  • ttc(200,20)==240
  • should be created
```

Ajouter temporairement une erreur dans le code de TvaService.tva (par exemple return -1) et visualiser ce message d'erreur :

Jasmine 4.6.1

X . .

3 specs, 1 failure, randomized with seed 26403

[Spec List](#) | Failures

TvaService > tva(200,20)==40

Expected 0 to be 40.

Restituer le bon code au sein de TvaService.tva() pour revenir au vert

Arrêter ng test via Ctrl-C

3.4. TP : test unitaire d'un composant simple (TvaComponent)

Ne rien ajouter dans tva.component.spec.ts (laisser dans l'état le code initialement généré par ng g c tva).

Lancer la commande suivante :

```
ng test --include=**/tva.component.spec.ts
```

Selon l'utilisation (ou pas) de pipe en version : 'fr' dans le composant tva , il faudra (ou pas) ajouter ceci dans le haut de `tva.component.spec.ts` :

```
import { registerLocaleData } from '@angular/common';
import localeFr from '@angular/common/locales/fr';
registerLocaleData(localeFr);
```

Au sein de **tva.component.spec.ts** ajouter et mettre au point (par correction éventuelle) cette première méthode de test :

//ancienne version de 2024 pour composant Tva sans signal en sans le "ZoneLesss" :

```
it('tva(200,20)=40 from model', () => {  
  component.ht=200;  
  component.tauxTvaPct=20;  
  component.onCalculerTvaTtc();//à ne pas oublier d'appeler si pas de dispatchEvent  
  fixture.detectChanges();  
  const compNativeElt = fixture.debugElement.nativeElement;  
  let spanTvaElt = compNativeElt.querySelector('#spanTva');  
  console.log("from model, tva:" + spanTvaElt.innerText);  
  expect(Number(spanTvaElt.innerText)).toBeCloseTo(40,2);  
  // .toBeCloseTo(expectedValue,precision_as_nb_decimal)  
});
```

//nouvelle version de 2026 pour composant Tva avec signaux en mode "ZoneLesss" :

```
it('tva(200,20)=40 from model', async () => {  
  component.ht.set(200);  
  component.tauxTvaPct.set(20);  
  fixture.changeDetectorRef.markForCheck();  
  //fixture.detectChanges();//not OK for ZoneLess mode  
  await fixture.whenStable(); //OK with async for ZoneLess mode  
  const compNativeElt = fixture.debugElement.nativeElement;  
  let spanTvaElt = compNativeElt.querySelector('#spanTva');  
  console.log("from model, tva:" + spanTvaElt.textContent);  
  expect(Number(spanTvaElt.textContent)).toBeCloseTo(40,2);  
  // .toBeCloseTo(expectedValue,precision_as_nb_decimal)  
});
```

//NB : **.textContent** fonctionne bien avec une version récente d'angular (ex : angular 21) mais **.innerText** ne fonctionne pas correctement avec **angular 21**.

Seulement après avoir mis au point le test précédent, on pourra tenter un second test plus complet de ce genre :

```
it('tva(200,10)=220 from IHM', async () => {  
  //Saisies de valeurs (via native_elements and DOM api):  
  const compNativeElt = fixture.debugElement.nativeElement;  
  let htInputElt = compNativeElt.querySelector("input[name='ht']");  
  htInputElt.value=200;  
  htInputElt.dispatchEvent(new Event('input'));  
  
  /* // Pour version simplifiée avec bouton et sans liste déroulante:  
  let tauxTvaPctInputElt = compNativeElt.querySelector("input[name='tauxTvaPct']");  
  tauxTvaPctInputElt.value=20;  
  tauxTvaPctInputElt.dispatchEvent(new Event('input'));  
  
  let calculButtonElt =  
    compNativeElt.querySelector("input[type='button'][value='calculer']");  
  //calculButtonElt.dispatchEvent(new Event('click'));  
  calculButtonElt.click(); */  
  
  // Pour version sans bouton et avec liste déroulante:  
  let tauxTvaPctSelectElt = compNativeElt.querySelector("select[name='tauxTvaPct']");  
  let optionElt = null;
```



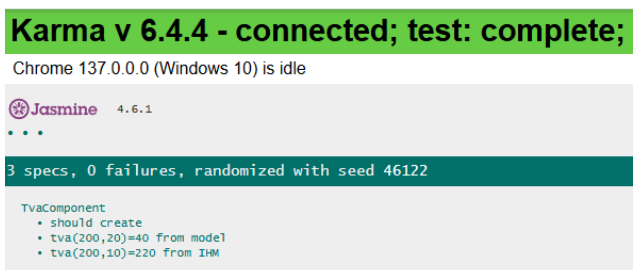
```

for(let opt of tauxTvaPctSelectElt.children){
  if(opt.textContent=="10%"){ optionElt=opt;
  }
}
console.log("from ihm, optionElt.textContent:" + optionElt.textContent
  + " , optionElt.value:" + optionElt.value);
tauxTvaPctSelectElt.value=optionElt.value;
//fixture.detectChanges(); //not OK for ZoneLess mode
await fixture.whenStable(); //OK with async for ZoneLess mode
console.log("from ihm, tauxTvaPctSelectElt.value:" + tauxTvaPctSelectElt.value);
tauxTvaPctSelectElt.dispatchEvent(new Event('change'));

await fixture.whenStable();
//Eventuelles vérifications des valeurs saisies et calculées dans le modèle:
expect(Number(component.ht())).toBe(200);
expect(component.tauxTvaPct()).toBe(10); //avec ou sans parenthèses si signaux ou pas
expect(component.ttc()).toBeCloseTo(220,2);

//Vérifications importantes des valeurs calculées dans la vue (template html):
let spanTtcElt = compNativeElt.querySelector('#spanTtc');
console.log("from IHM, res:" + spanTtcElt.textContent);
expect( Number( spanTtcElt.textContent ) ).toBeCloseTo(220,2);
});
});

```

Résultats dans fenêtre "Karma" pour angular 17,18,19	Résultats dans console texte "Vitest" de angular 21
 Karma v 6.4.4 - connected; test: complete; Chrome 137.0.0.0 (Windows 10) is idle Jasmine 4.6.1 3 specs, 0 failures, randomized with seed 46122 TvaComponent • should create • tva(200,20)=40 from model • tva(200,10)=220 from IHM	from IHM, res:220 ✓ my-app src/app/basic/tva/tva.component.spec.ts (3 tests) 105ms ✓ TvaComponent (3) ✓ should create 55ms ✓ tva(200,20)=40 from model 40ms ✓ tva(200,10)=220 from IHM 9ms Test Files 1 passed (1) Tests 3 passed (3) Start at 09:19:33 Duration 1.22s (transform 79ms, setup 280ms, import 73ms, tests 105ms, environment 624ms)

NB : s'il faut effectuer un test de CalculatriceComponent et si le constructeur de CalculatriceComponent a besoin du service ActivatedRoute , on est alors obligé d'ajouter un mock (implémentation fournie simplifiée de ActivatedRoute) via ce genre de paramétrage au sein de calculatrice.component.spec.ts :

```

beforeEach(async () => {
  await TestBed.configureTestingModule({
    imports: [CalculatriceComponent],
    providers: [ {
      provide: ActivatedRoute, useValue: { params: of({mode: 'simple'}) }
    } ]
  }) .compileComponents();
}

```